

Linux Production Operations & Self-Healing System

Automated First-Level Incident Response, System Monitoring, and Controlled Recovery

Cloud Support / Junior DevOps Project Documentation

1. Overview & Problem Statement

In production Linux environments, servers require continuous checks for resource usage, running services, network reachability, security policies, and critical system logs. Manual monitoring is time-consuming and can delay first-level incident response. This project automates first-level health checks using Bash scripts, detects failures, safely recovers known services, and logs all outcomes.

2. Key Project Objectives

- **Automated Diagnostics:** Execute continuous first-level system health checks across production nodes.
- **Failure Detection:** Identify outages in critical daemons, network ports, high resource utilization, and log errors.
- **Controlled Self-Healing:** Automatically restart known, safe service failures without human intervention.
- **Verification & Audit Logging:** Confirm post-recovery service status and produce audit-ready health reports.

3. Operational Architecture Workflow



4. Project Directory Structure

```
/opt/linux-ops/
├── scripts/
│   └── linux_ops.sh      # Core health monitoring & self-healing engine
├── config/
│   └──                  # Service thresholds and operational parameters
├── logs/
│   └── linux_ops.log     # System execution & incident recovery logs
├── reports/
│   └── health_report.txt # Consolidated system health status output
```

5. Scope of Monitoring Coverage

Category	Monitored Components & Operational Tools
System Resources	CPU Usage, RAM Allocation, Disk Partition Consumption
Core Daemons	Apache (httpd), BIND DNS (named), FTP (vsftpd), NTP (chronyd)
Network & Ports	Port 80 (HTTP), Port 53 (DNS), Port 21 (FTP), Default Gateway & Internet Ping
Security & Storage	Firewalld status, SELinux mode (Enforcing), /mydata ownership & permissions
Logs & Diagnostics	Recent kernel and system level error logs via journalctl

6. Safety Guardrails & Operational Constraints

To ensure system stability, the self-healing engine enforces strict recovery boundaries:

- **No Destructive Actions:** Disallows unverified or automated file deletions.
- **No Destructive Permissions:** Restricts automatic aggressive file permission overrides.

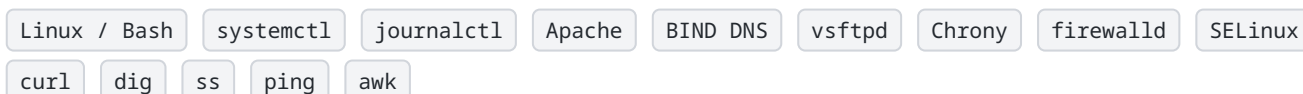
- **Security Controls Retained:** Keeps SELinux in Enforcing mode and never flushes Firewallld rules.
- **Manual Escalation:** High-risk failures are flagged for review and logged without automated intervention.

7. Incident Simulation & Demonstration

A real-world service outage was simulated to test the self-healing pipeline:

- **Trigger:** Apache web server (httpd) service was manually stopped on `server1`.
- **Detection:** The script flagged the service as `INACTIVE` using `systemctl`.
- **Recovery:** Triggered a controlled service restart.
- **Verification:** Confirmed endpoint availability using `curl` HTTP requests.
- **Log Output:** Logged status as `RECOVERED` in `linux_ops.log`.

8. Key Technologies & Tools



9. Interview Pitch & Future Enhancements

Interview Pitch: "I developed a Bash-based Linux production operations script that automates first-level health checks across resources, network ports, security policies, and system daemons. It features a safety-focused self-healing mechanism that automatically recovers known failed services and verifies recovery with active health checks."

Future Enhancements: Integration with Email/Slack alerting, centralized logging (Elasticsearch/CloudWatch), JSON dashboard metrics, and Jenkins CI/CD pipeline triggering.